Sprint 7 — Adapter Pattern

Class 2

Lesson 7 — Adapter vs Other Patterns

This is the interview question we struggle with the most.

Adapter

Purpose: Make two incompatible interfaces work together.

Application -> PaymentGateway -> TrueMoneyAdapter -> TrueMoney SDK

Application doesn't know TrueMoney.

Facade

Purpose: Hide complexity.

Example

Instead of

InventoryService
PaymentService
ShippingService
EmailService
InvoiceService

We expose

OrderFacade.placeOrder()

One method.

Many operations.

Decorator

Purpose: Add new behavior without modifying the object.

File
CompressionDecorator
EncryptionDecorator
LoggingDecorator

Same interface.

More features.

Proxy

Purpose: Control access.

Application -> SecurityProxy -> RealService

Proxy decides

Allow?

Reject?

Cache?

Log?

Interview Trick

Which pattern changes the interface?

Answer:Adapter.

Lesson 8 — Java Examples

Many developers use Adapter every day without realizing it.

Example 1

InputStreamReader

```
InputStream input = ...

Reader reader =
        new InputStreamReader(input);
```

Question

Why?

Because

InputStream -> InputStreamReader -> Reader

InputStreamReader adapts byte streams into character streams.

Perfect Adapter.

Example 2

Arrays.asList()

```
String[] names = { "A","B"};

List<String> list = Arrays.asList(names);
```

Array -> List interface

Different interfaces.

Example 3

Collections.enumeration()

```
List<String> list = ...

Enumeration<String> e =
        Collections.enumeration(list);
```

List -> Enumeration

Adapter.

Lesson 9 — Spring Examples

Spring MVC

DispatcherServlet -> HandlerAdapter -> Controller

Spring does not know which controller type.

HandlerAdapter translates.

This is literally Adapter Pattern.

WebFlux

ServerRequest -> HandlerFunctionAdapter -> HandlerFunction

HttpMessageConverter

JSON -> Java Object -> JSON

Different representations.

Spring uses adapters to convert between them.

Lesson 10 — Enterprise Example

Payment System

Our application

```
public interface PaymentGateway {

    PaymentResult pay(
            PaymentRequest request);

}
```

Implementations:

TrueMoneyAdapter

PayPalAdapter

ABAAdapter

ACLEDAAdapter

WingAdapter

Business code Never changes.

Lesson 11 — Anti-Corruption Layer (ACL)

This is a real enterprise concept from Domain-Driven Design.

Many of us have never heard of it.

Suppose TrueMoney SDK returns TrueMoneyPaymentStatus.SUCCESS

Our business should not use TrueMoneyPaymentStatus

Instead Adapter converts TrueMoney -> PaymentStatus.SUCCESS

Our business model remains independent.

This protective boundary is called an **Anti-Corruption Layer (ACL)**.

Another example

Hotel API returns ROOM_READY

Our application uses AVAILABLE

Adapter converts it.

The rest of our application never depends on the hotel's terminology.

Lesson 12 — Common Mistakes

Mistake 1

Business logic inside Adapter.

Wrong

if (amount > 1000) {

discount...

}

Adapter should only translate.

Mistake 2

Adapter modifying SDK.

Never.

SDK belongs to another company.

Mistake 3

Leaking SDK objects.

Wrong: public StripeResponse pay(...)

Correct: public PaymentResult pay(...)

Never expose third-party models.

Mistake 4

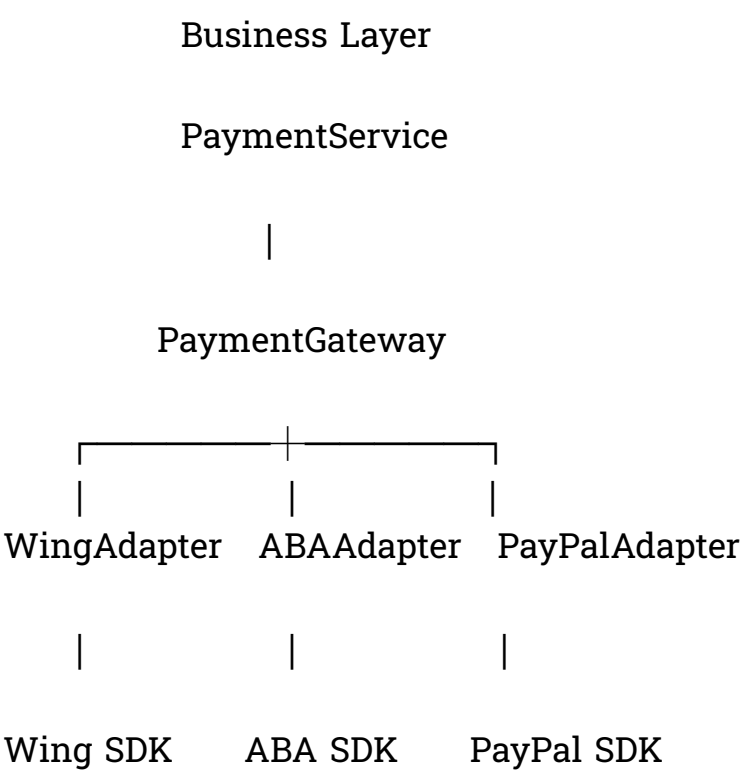
One giant Adapter.

Wrong

EverythingAdapter

Create focused adapters.

Enterprise Architecture



This is exactly how enterprise systems support multiple vendors.

Homework

Build File Storage Platform.

Interface StorageService

Implement

LocalStorageAdapter

MinioStorageAdapter

S3StorageAdapter

GoogleCloudStorageAdapter

Without changing MediaService

Bonus

Add Azure Blob Storage.

Interview Questions

1 What problem does Adapter solve?

Expected answer

It converts one interface into another so existing code can work with incompatible classes without modification.

2 Difference between Adapter and Facade?

Adapter	Facade
Changes interface	Simplifies interface
Solves incompatibility	Reduces complexity

3 Difference between Adapter and Decorator?

Adapter	Decorator
Changes interface	Adds behavior
Focuses on compatibility	Focuses on extensibility

4 Why is Object Adapter preferred?

Because Java supports only single inheritance, and composition provides greater flexibility.

5 Give three Java examples.

Expected answers

- InputStreamReader
- Arrays.asList()
- Collections.enumeration()

6 Can Adapter wrap REST APIs?

Yes.

It can also wrap:

- SOAP services
- Database drivers
- Payment SDKs
- Cloud SDKs
- Legacy systems

7 What is an Anti-Corruption Layer?

An Adapter layer that prevents third-party models and terminology from leaking into your domain model.

8 Can multiple adapters implement the same interface?

Yes.

That's exactly how systems support multiple vendors while keeping business code unchanged.

Summary of Adapter Pattern

Component	Responsibility
Client	Uses the target interface
Target	The interface expected by the client
Adapter	Converts between interfaces
Adaptee	Existing class or external SDK
Object Adapter	Uses composition (recommended in Java)
Class Adapter	Uses inheritance (rare in Java)

Real Examples We Recognize

After this class, we should realize they've already seen Adapter Pattern in many places:

- Java I/O (InputStreamReader)
- Spring MVC (HandlerAdapter)
- Spring WebFlux (HandlerFunctionAdapter)
- Jackson (HttpMessageConverter)
- Payment integrations (Stripe, PayPal, ABA, ACLEDA)
- Cloud storage (MinIO, AWS S3, Google Cloud Storage)
- Hotel/PMS integrations
- Legacy SOAP services